

CUDA implementation of the BFAST algorithm

PMPH 2022 project presentation
Group KTLPC

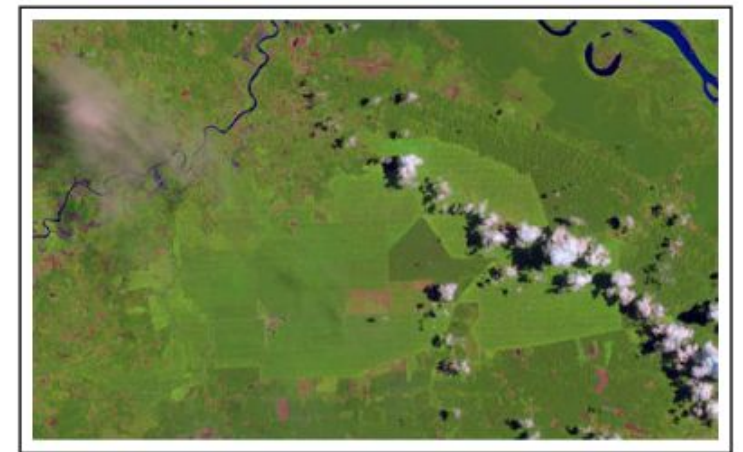
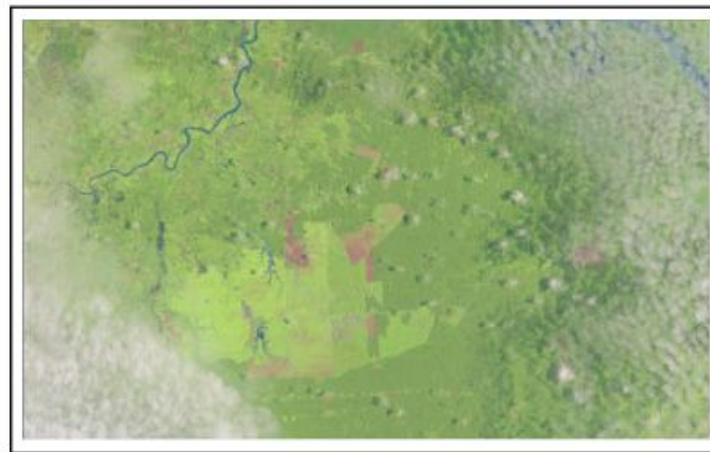
10.11.2022

UNIVERSITY OF COPENHAGEN



The problem

- Satellite data – we want to detect deforestation
- BFAST algorithm



Project goal and execution

- “efficient CUDA implementation for a variation of the BFAST algorithm”
- Given:
 - Futhark implementation
 - 2 sets of data for testing: peru and sahara

What had to be done:

- Parsing data
- Sequential implementation + validation against futhark version
- Parallel implementation + validation against sequential version
- Optimizing kernels
- Measuring & reporting performance

Parsing the data

- Data provided was in the futhark format
- `load.fut`
- `read_data.cu`
 - `parse_primitive`
 - `read_primitive`
 - `read_arr`
- How to run
 - `zcat <path to data> | ./load | ./bfast_main`

Sequential implementation

- Served as a wireframe for the parallel implementation and a comparison point for the GPU implementation
- Implemented with sequential loops
- Results were used as a source of truth for checking correctness of the GPU implementation

CUDA implementation – overview

- Levels of optimizations – naive and optimized
- Priorities – kernel 2 and 3
- What techniques were used
 - Registered tiling, shared memory
 - Described where exactly with each kernel
- How we measured performance
 - Time, Gbit/s and GFLOPS
 - N runs
 - gettimeofday (i.e. not the cuda events for measuring)

Validation

- Validation of sequential version
- Validation of parallel version
- Comparing floats
 - Absolute epsilon = 0.005
 - Relative epsilon = 0.02
- Counting wrong values
- Correctness of our implementation

```
...  
Call CUDA Kernel 8 =====  
'breaks' and 'h_breaks' differ at 2 out of 111556 positions (0.0018%)  
they first differ at position 12216: -1 vs. 115  
means validates
```

Kernel 1

- Goal of the kernel: create interpolation matrix X
 - Two versions: trend and no trend, mutually exclusive
- Also part of this section:
 - Prepare X_h , X_{th} , Y_h for the next kernel
 - Slice matrix kernel
 - Transpose kernel (from weekly 3)

```

let K = 2*k + 2
let X = mkX K f -- [K] [N] f32 -- ker 1
let XT = transpose X
let (X[:,n], XT[:,n]) = (X[:, :n], XT[:, n, :]) in
map (λy → let y[:,n] = y[:, n]

```

```

let k2p2 = 2*k + 2
let k2p2' = i64.i32 <|
  if trend > 0 then k2p2 else k2p2-1
let X = opaque <| if trend > 0
  then mkX_with_trend k2p2' freq mappingindices
  else mkX_no_trend k2p2' freq mappingindices
let zero = r32 <| i32.i64 <| (N*N + 2*N + 1) / (N+1) - N - 1
let Xt = opaque <| map (map (+zero)) (copy (transpose X))
let Xh = (X[:, :n])
let Xth = (Xt[:, n, :])
let Yh = (images[:, :n])

```



```
let mkX_with_trend [N] (k2p2: i64) (f: f32) (mappingindices: [N]i32): [k2p2][N]f32 =  
  map (\ i -> let i = i32.i64 i in map (\ind ->  
    if i == 0 then 1f32  
    else if i == 1 then r32 ind  
    else let (i', j') = (r32 (i / 2), r32 ind)  
          let angle = 2f32 * f32.pi * i' * j' / f  
          in if i % 2 == 0 then f32.sin angle  
             else f32.cos angle  
    ) mappingindices  
  ) (iota k2p2)
```

```
void mkX_with_trend(int k2p2, float f, int* mapping_indecies, int N, float* result){  
  for(int i = 0; i < k2p2; i++) {  
    for(int j = 0; j < N; j++) {  
      if(i == 0) { result[i*N+j] = 1.0; }  
      else if (i == 1) { result[i*N+j] = (float)mapping_indecies[j];}  
      else {  
        float i1 = (float)(i/2);  
        float j1 = (float)(mapping_indecies[j]);  
        float angle = 2.0 * M_PI * i1 * j1/f;  
        if (i%2 == 0) { result[i*N+j] = sinf(angle); }  
        else { result[i*N+j] = cosf(angle); }  
      }  
    }  
  }  
}
```

```
void mkX_with_trend(int k2p2, float f, int* mapping_indecies, int N, float* result){
    for(int i = 0; i < k2p2; i++) {
        for(int j = 0; j < N; j++) {
            if(i == 0) { result[i*N+j] = 1.0; }
            else if (i == 1) { result[i*N+j] = (float)mapping_indecies[j];}
            else { ...
        }
    }
}
```

```
__global__ void mkX_with_trend_ker(int K, int N, float f, int* mapping_indecies, float* result){
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    int i = gid / N;
    int j = gid % N;
    if(gid > (N*K)) return;
    result[gid] = 0.0;
    if(i == 0) result[gid] = 1.0;
    else if (i == 1) result[gid] = (float)mapping_indecies[j];
    else { ...
}
```

Matrix slice

```
let Xh = (X[:, :n])
```

```
// X[:, :n]
for (int i = 0; i < k2p21; i++) {
    for (int j = 0; j < n; j++) {
        Xh[i*n+j] = X[i*N+j];
    }
}
```

```
__global__ void sliceMatrixKernel(int m, int N, int n, float* M_in, float* M_out){
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    if(gid > (m * n)) return;
    int i_row = gid / n;
    int i_col = gid % n;
    M_out[gid] = M_in[i_row * N + i_col];
}
```

Kernel 1 performance

- Peru
 - 9293 vs 1021 microseconds
 - 9x speedup
- Sahara
 - 10963 vs 1278 microseconds
 - 8x speedup

Kernel 2 Overview

- Goal: Create Xsqr by matrix multiplication and filtering
- Call of naive version:

```
int dimx = m;
int dimy = (K + 16 - 1) / 16;
int dimz = (K + 16 - 1) / 16;
dim3 grid (dimx, dimy, dimz);
dim3 block (min(16,K), min(16,K), 1);
for(int i = 0; i < GPU_RUNS; i++) {
    kernel2_naive<float,16> <<< grid, block >>>
                                (d_Xh, d_Xth, d_Yh, m, n, K, d_Xsqr_naive);
}
```

Kernel 2 Naive

- Naive implementation:

```
forall(i=0; i<M; i++) {           // parallel
    forall(j1=0; j1<K1; j1++) {    // parallel
        forall(j2=0; j2<K2; j2++) { // parallel
            float acc = 0.0;
            forseq(q=0; q<n; q++) {
                float a = A[j1,q], b = B[q,j2], ab=a*b;
                acc += ab * (1.0 - isnan(Y[i,q]));
            }
             $\bar{M}[i, j_1, j_2] = \text{acc};$ 
        }
    }
}
```

(a) Naive Imperative Version

```
template<class ElTp, int T>
__global__ void kernel2_naive(ElTp* Xh, ElTp* Xth, ElTp* Yh, int m, int n, int K, ElTp* Xsqr) {
    int i    = blockIdx.x;
    int gidy = blockIdx.y * blockDim.y + threadIdx.y; //j1
    int gidx = blockIdx.z * blockDim.x + threadIdx.x; //j2

    if(gidx >= K || gidy >= K) return;
    float acc;

    acc = 0.0;

    for(int q = 0; q < n; q++) {
        float a = Xh[gidy * n + q];
        float b = Xth[q * K + gidx];
        float ab = a*b;
        acc += ab * (1.0-(float)isnan(Yh[i*n + q]));
    }
    Xsqr[i*K*K + gidy*K + gidx] = acc;
}
```


Kernel 2 Optimized

- Register-tiled
- Uses shared memory
- Results in: $R \times$ fewer accesses to arrays A, B and Y
- Yh is transposed to Yht

```
int T = min(16,K);
int dimz = (m + R - 1) / R;
int dimy = (K + T - 1) / T;
int dimx = (K + T - 1) / T;
dim3 grid (dimx, dimy, dimz);
dim3 block (T, T, 1);

for(int i = 0; i < GPU_RUNS; i++) {
    kernel2_mat_mult<float,TILE, R> <<< grid, block >>>
        (d_Xh, d_Xth, d_Yht, m, n, K, d_Xsqr);
}
cudaDeviceSynchronize();
```

Kernel 2 Optimized

```

YT = transpose(Y);
forall(ii=0; ii<M; ii+=R) {           // grid.z
    forall(jj1=0; jj1<K1; jj1+=T1) {   // grid.y
        forall(jj2=0; jj2<K2; jj2+=T2) { // grid.x
            forall(j1=jj1; j1<min(jj1+T1,K1);
                j1++) {                // block.y
                forall(j2=jj2; j2<min(jj2+T2,K2);
                    j2++) {            // block.x
                    float Yshq[R];    // shared memory
                    float acc[R];      // registers
                    for(i=0; i<R; i++) // fully unroll
                        acc[i] = 0.0;
                    float a, b, ab, y;
                    forseq(q=0; q<n; q++) { barrier;
                        float a=A[j1,q], b=B[q,j2], ab=a*b;
                        // collective copy global-to-shared
                        Yshq[0:R] = YT[q,ii:min(ii+R,M)];
                        barrier; // block-level synch
                        forseq(i=0; i<R; i++){ // fully unroll
                            if(ii+i < M)
                                acc[i] += ab*(1.0-isnan(Yshq[i]));
                        } }
                    forseq(i=0; i<R; i++) // fully unroll
                        if (ii+i<M) M̄[ii+i,j1,j2] = acc[i];
                } } } } }

```

```

template<class ElTp, int T, int R>
__global__ void
kernel2_mat_mult(ElTp* Xh, ElTp* Xth, ElTp* Yth, int m, int n, int K, ElTp* Xsqr) {

    unsigned int ii = blockIdx.z * R;           //grid z
    unsigned int jj1 = blockIdx.y * min(T,K);   //grid y
    unsigned int jj2 = blockIdx.x * min(T,K);   //grid x
    unsigned int j1 = jj1 + threadIdx.y;        //Block y
    unsigned int j2 = jj2 + threadIdx.x;        //Block x
    unsigned int tid = threadIdx.y * blockDim.x + threadIdx.x; //Thread id inside block

    __shared__ ElTp Ysh[R];
    float acc[R];

    #pragma unroll
    for(int i = 0; i < R; i++) {
        acc[i] = 0.0;
    }

    float a, b, ab;
    for(int q = 0; q < n; q++) {
        __syncthreads();
        a = Xh[j1*n + q];
        b = Xth[q * K + j2];
        ab = a*b;

        if (tid < R && ii + tid < m) {
            Ysh[tid] = Yth[q*m + (ii + tid)];
        }
        __syncthreads();
    }
}

```


Kernel 2 Optimized

```

YT = transpose(Y);
forall(ii=0; ii<M; ii+=R) {           // grid.z
    forall(jj1=0; jj1<K1; jj1+=T1) {    // grid.y
        forall(jj2=0; jj2<K2; jj2+=T2) { // grid.x
            forall(j1=jj1; j1<min(jj1+T1,K1);
                j1++) {                 // block.y
                forall(j2=jj2; j2<min(jj2+T2,K2);
                    j2++) {             // block.x
                    float Yshq[R];      // shared memory
                    float acc[R];        // registers
                    for(i=0; i<R; i++)   // fully unroll
                        acc[i] = 0.0;
                    float a, b, ab, y;
                    forseq(q=0; q<n; q++) { barrier;
                        float a=A[j1,q], b=B[q,j2], ab=a*b;
                        // collective copy global-to-shared
                        Yshq[0:R] = YT[q,ii:min(ii+R,M)];
                        barrier;          // block-level synch
                        forseq(i=0; i<R; i++) { // fully unroll
                            if(ii+i < M)
                                acc[i] += ab*(1.0-isnan(Yshq[i]));
                        } }
                    forseq(i=0; i<R; i++) // fully unroll
                        if (ii+i<M) M̄[ii+i,j1,j2] = acc[i];
                } } } } }

```

```

#pragma unroll
for(int i = 0; i < R; i++) {
    if(ii + i < m && !isnan(Ysh[i])) {
        acc[i] += ab;
    }
}

#pragma unroll
for(int i = 0; i < R; i++) {
    if(ii + i < m && j2 < K && j1 < K) {
        Xsqr[(ii+i)*K*K + j1*K + j2] = acc[i];
    }
}

```

Kernel 2 Improvements

Naive tested on the peru dataset	Gigaflops	microseconds
GPU04 (2080 ti)	143.977	22414
futharkhpa03fl.unicph.d omain (A100)	1055.295	3162

CPU in microsecs
108541

Optimized tested on the peru dataset	Gigaflops	microseconds	Improvement over naive (comparing gigaflops)	Improvement over cpu
GPU04 (2080 ti)	5316.461	816	36.926x	133.016x
futharkhpa03fl.unicph.d omain (A100)	7030.701	563	6.66x	192.790x
futharkhpa03fl.unicph.d omain (A100) R = 22	7968.128	502	7.551x	216.217x

Kernel 3

- CPU Kernel: Sequential processing of m matrices
- GPU Kernel Goals:
 - Divide and Conquer: optimal number of blocks and threads
 - Efficient Memory access: minimize global memory access
 - Parallelisation: exploit parallelism
- Divide and Conquer:
 - Blocks: m blocks one per matrix
 - Threads: $K * (2*K)$ threads one per matrix element
- Efficient Memory:
 - Copy necessary data from global to shared
- Parallelisation:
 - Implement the algorithm proposed in the paper

```
float[M][K][K] batchMatInv(float A[M][K][K]) {
    float A-1[M][K][K]; // global memory result
    forall i = 0...M-1 { // grid.x
        float Ash[K][2*K]; // shared memory
        // Pad A with identity matrix to the right
        forall k1 = 0...K-1 { // block.y
            forall k2 = 0...2*K-1 { // block.x
                if (k2<K) { Ash[k1,k2] = A[i,k1,k2]; }
                else { Ash[k1,k2] = (k2 == K+k1); }
                barrier; // block-level synch
            } } // end forall k1/2
        // Gauss-Jordan Elimination:
        forseq q = 0...K-1
            float vq = Ash[0,q]
            forall k1 = 0...K-1 { // block.y
                forall k2 = 0...2*K-1 { // block.x
                    float tmp = 0.0;
                    if (vq == 0.0) tmp = Ash[k1,k2];
                    else {
                        float x = Ash[0,k2] / vq;
                        if(k1 == K-1) tmp = x;
                        else tmp = Ash[k1+1,k2] -
                            Ash[k1+1,q] * x;
                    }
                }
                barrier; // block-level synch
                Ash[k1,k2] = tmp;
                barrier; // block-level synch
            } } // end forall k1/2
        } // end forseq q
        // collective copy shared-to-global mem:
        A-1[i, 0:K, 0:K] = Ash[0:K, K:2*K];
    } } // end forall i
}
```

Fig. 5: Batched Matrix Inversion: Loop-Based Specification

Kernel 3 (Setup)

- Initialize shared memory (size was defined at kernel launch time)
- Compute global address for reads and writes
- Identify row and col of each thread

```
__global__ void
kernel_3(float* Xsqr, float* Xinv, int K, int m) {
    extern __shared__ float A_sh1[];
    volatile float* A_sh = (volatile float*) A_sh1; // size K * (K*2)

    if (blockIdx.x < m) {
        int length_of_matrix = K*K; // K x K matrix
        // blockIdx.x: 0..m
        int start_index = blockIdx.x * length_of_matrix;

        // A_sh one matrix with adjoined identity matrix in shared memory
        // threadIdx: 0...(K*(2*K))
        int k1 = threadIdx.x / (2*K); // row of A_sh
        int k2 = threadIdx.x % (2*K); // col of A_sh
```

Kernel 3 (Build Matrix/Compute Inversion)

- Build matrix in parallel, each thread handles one element
- Calculate inversion
 - Keep outer sequential loop
 - Internal loop can be replaced $K * (2*K)$ parallel accesses
 - Synchronize result after each iteration

```
let gauss_jordan [nm] (n:i32) (m:i32) (A: *[nm]f32): [nm]f32 =
  loop A for i < n do
    let v1 = A[i]
    let A' = map (\ind -> let (k, j) = (ind / m, ind % m)
      in if v1 == 0.0 then A[k*m+j] else
        let x = (A[j] / v1) in
          if k < n-1 -- Ap case
            then ( A[(k+1)*m+j] - A[(k+1)*m+i] * x )
            else x      -- irow case
        ) (map i32.i64 (iota nm))
    in scatter A (iota nm) A'
```

```
let mat_inv [n] (A: [n][n]f32): [n][n]f32 =
  let m = 2*n
  let nm = n*m
  -- Pad the matrix with the identity matrix.
  let Ap = map (\ind -> let (i, j) = (ind / m, ind % m)
    in if j < n then ( A[i,j] )
      else if j == n+i
        then 1.0
        else 0.0
    ) (iota nm)
  let Ap' = gauss_jordan (i32.i64 n) (i32.i64 m) Ap
```

```
// build matrix
if (k2 < K) {
  A_sh[k1 * (2*K) + k2] = Xsqr[start_index + k1 * K + k2];
}else{
  if(k2 == (K + k1)) {A_sh[k1 * (2*K) + k2] = 1.0f;}
  else{A_sh[k1 * (2*K) + k2] = 0.0f;}
}
__syncthreads();

// Calculate Inversion
for(int q = 0; q < K; q++) {
  float vq = A_sh[q]; float tmp = 0.0f;
  if (vq == 0.0f) {tmp = A_sh[k1 * (2*K) + k2];}
  else {float x = A_sh[k2] / vq;
    if (k1 == (K-1)) {tmp = x;}
    else {
      tmp = (A_sh[(k1+1) * (2*K) + k2] -
        A_sh[(k1+1) * (2*K) + q] * x);
    }
  }
  __syncthreads();
  A_sh[k1 * (2*K) + k2] = tmp;
  __syncthreads();
}
```


Kernel 3 (Write to global/Wrap up)

- Write the right side of the matrix to global memory
- Reads from global: $K \times K$, Writes to global: $K \times K$
- Naive version is identical with the minor difference the array `A_sh` is passed as a parameter

```
in ((unflatten n m Ap')[0:n,n:m]) :> [n][n]f32
```

```
if((k1 < K) && (k2 < K)) {  
    Xinv[start_index + (k1 * K + k2)] = A_sh[k1 * (2*K) + k2 + K];  
}
```

Kernel 3 (Performance)

- The speed up between the naive and optimized version is minor $\sim 1.08x$ (depends on hardware)
- But we achieved a significant speed up compared to CPU $\sim 179x$ on Peru and $\sim 131x$ on Sahara

	Naive tested on the peru dataset	Gigaflops	microseconds	GB/s
Naive	futharkhpa03fl.unicph.domain (A100)	810.17	423	202.54
Optimized	futharkhpa03fl.unicph.domain (A100)	876.47	391	219.12
Paper	Hardware defined in Paper	~ 450	~ 730	

CPU in microseconds on Peru

73066

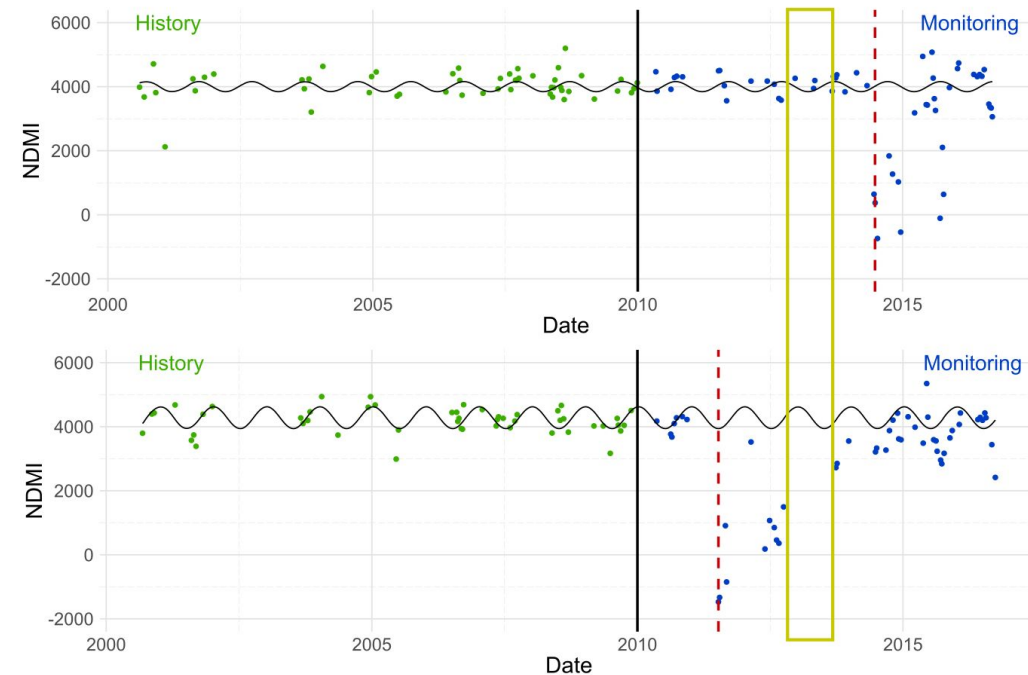
Kernel 4

- Goal of the kernel: compute model + predicted values for each pixel $\hat{y}$
- Outer parallelism (map across pixels) translated into 2 MM Multiplications and “m” many MV Multiplication
- Naive version: Threads according to dimensions
- Optimized versions similar to weekly 3
 - Block-tiling version
- Initial Problem: figuring out correct dimensions (Transpose needed)

```

map (λy → let  $\bar{y}_{[:n]}$  =  $y[:n]$ 
           . . .
           let  $\bar{\beta}_0$  = mvMulFilt  $\mathbf{X}_{[:,n]}$   $y[:n]$ 
           let  $\bar{\beta}$  = mvMul  $\mathbf{X}^{sq\tau^{-1}}$   $\bar{\beta}_0$ 
           let  $\hat{y}$  = mvMul  $\mathbf{X}^T$   $\bar{\beta}$ 
           . . .
        ) Y

```



Kernel 4 (Details)

map ($\lambda y \rightarrow \text{let } \bar{y}_{[:n]} = y[:n]$

...

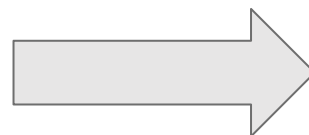
let $\bar{\beta}_0 = \text{mvMulFilt } \mathbf{X}_{[:,n]} \mathbf{y}_{[:n]}$

let $\bar{\beta} = \text{mvMul } \mathbf{X}^{sqr^{-1}} \bar{\beta}_0$

let $\hat{y} = \text{mvMul } \mathbf{X}^T \bar{\beta}$

...

) **Y**



$$\bar{B}_0 = \text{mmMulFilt } \mathbf{X}_{[:,n]} \mathbf{Y}_{[:,n]}^T$$

$$\bar{\mathbf{B}} = \text{mvMul_m_times } \mathbf{X}_m^{sqr^{-1}} \bar{B}_0^T$$

$$\hat{\mathbf{Y}} = \text{mmMul } \mathbf{X}^T \bar{\mathbf{B}}$$

$$\bar{\beta}_0 \in \mathbb{R}^K, \mathbf{X}_{[:,n]} \in \mathbb{R}^{K \times n}, \mathbf{y}_{[:n]} \in \mathbb{R}^n,$$

$$\bar{\beta} \in \mathbb{R}^K, \mathbf{X}^{sqr^{-1}} \in \mathbb{R}^{K \times K},$$

$$\hat{y} \in \mathbb{R}^N, \mathbf{X}^T \in \mathbb{R}^{N \times K},$$

$$\mathbf{Y} \in \mathbb{R}^{m \times N}$$

$$\bar{B}_0 \in \mathbb{R}^{K \times m}, \mathbf{X}_{[:,n]} \in \mathbb{R}^{K \times n}, \mathbf{Y}_{[:,n]}^T \in \mathbb{R}^{n \times m},$$

$$\bar{\mathbf{B}} \in \mathbb{R}^{K \times m}, \mathbf{X}_m^{sqr^{-1}} \in \mathbb{R}^{m \times K \times K},$$

$$\hat{\mathbf{Y}} \in \mathbb{R}^{N \times m}, \mathbf{X}^T \in \mathbb{R}^{N \times K},$$

Kernel 4 (Details)

```
__global__ void
mvMul_m_times(f32* X_sqr_inv, f32* beta_0, f32* beta, i32 K, i32 m) {
    int gidx = blockIdx.x*blockDim.x + threadIdx.x; // 0..K
    int gidy = blockIdx.y*blockDim.y + threadIdx.y; // 0..m

    if( (gidx >= K) || (gidy >= m) ) return;

    f32 acc = 0.0f;
    for(int j = 0; j < K; j++) {
        acc += X_sqr_inv[gidy*K*K + gidx * K + j] * beta_0[gidy * K + j];
    }
    beta[gidy*K + gidx] = acc;
}
```

	Ker4
CPU in μ -sec	50254
GPU in μ -sec	571
Speed-Up	88

Peru dataset on A100

	Ker4
CPU in μ -sec	24144
GPU in μ -sec	575
Speed-Up	41

Sahara dataset on A100

Kernel 5

- Goal of the kernel
 - Compute y_errors
 - Filter NaN values
- Loop analysis
- Naïve vs. Optimized kernel

Algorithm 1 Kernel 5

```
1:  $sum \leftarrow 0$ 
2: for  $i = 0, \dots, N$  do
3:    $\bar{\mathbf{r}}[i] \leftarrow \text{NaN}$ 
4:    $\bar{\mathbf{I}}[i] \leftarrow 0$ 
5:   if  $\mathbf{y}[i]$  is not NaN then
6:      $\bar{\mathbf{r}}[sum] \leftarrow \mathbf{y}[i] - \hat{\mathbf{y}}[i]$ 
7:      $\bar{\mathbf{I}}[sum] \leftarrow i$ 
8:      $sum \leftarrow sum + 1$ 
9:   end if
10:   $\bar{N} \leftarrow sum$ 
11: end for
```

Kernel 5

- Goal of the kernel
 - Compute `y_errors`
 - Filter NaN values
- Loop analysis
- Naïve vs. Optimized kernel
- Speedups

Benchmark results (sahara)		
	Time (μ-sec)	Speedup
CPU	34729	1.00x
Naïve	8600	4.04x
Optimized	619	56.11x

Kernel 6 Overview

- Goal: To calculate nss, hs and sigmas
- How to call:

```
dim3 grid(m, 1, 1);
dim3 block(n, 1, 1);
for (int run=0; run<GPU_RUNS; run++) {
    kernel6<f32, i32> <<< grid, block, 2*n*sizeof(f32) >>>(d_Yh, d_y_errors, m, n, N, k2p2, hfrac, d_nss, d_hs, d_sigmas);
}
cudaDeviceSynchronize();
```


Kernel 6



```
let (hs, nss, sigmas) = opaque <| unzip3 <|
  map2 (\yh y_error ->
    let ns      = map (\ye -> if !(f32.isnan ye) then 1i32 else 0i32) yh
        |> reduce (+) 0i32
    let sigma = map i32.i64 (iota n)
        |> map (\i -> if i < ns then y_error[i] else 0.0f32)
        |> map (\a -> a*a ) |> reduce (+) 0.0f32
    let sigma = f32.sqrt ( sigma / (r32 (ns-k2p2)) )
    let h      = t32 ( (r32 ns) * hfrac )
    in  (h, ns, sigma)
  ) Yh y_errors
```

```
template<class ElTp_default, class ElTp_nss_hs>
__global__ void kernel6(ElTp_default* Yh, ElTp_default* y_errors,           //in arrays
                        i32 m, i32 n, i32 N, i32 k2p2, f32 hfrac,           //in constants
                        ElTp_nss_hs* nss, ElTp_nss_hs* hs, ElTp_default* sigmas) { //out arrays

  extern __shared__ ElTp_default shmem[];
  volatile ElTp_default* Y_sh = shmem;
  volatile ElTp_default* Y_err_sh = shmem + n;

  Y_sh[threadIdx.x] = Yh[blockIdx.x*n + threadIdx.x];
  Y_err_sh[threadIdx.x] = y_errors[blockIdx.x * N + threadIdx.x];

  __syncthreads();

  Y_sh[threadIdx.x] = isnan(Y_sh[threadIdx.x]) ? 0.0 : 1.0;

  __syncthreads();

  ElTp_default tmp = scanIncBlock< Add<ElTp_default> >(Y_sh, threadIdx.x);
  __syncthreads();
  Y_sh[threadIdx.x] = tmp;

  __syncthreads();

  ElTp_nss_hs red_res = (ElTp_nss_hs) Y_sh[n-1];
  __syncthreads();
```

Kernel 6

```
let (hs, nss, sigmas) = opaque <| unzip3 <|
  map2 (\yh y_error ->
    let ns      = map (\ye -> if !(f32.isnan ye) then 1i32 else 0i32) yh
    |> reduce (+) 0i32
    let sigma = map i32.i64 (iota n)
    |> map (\i -> if i < ns then y_error[i] else 0.0f32)
    |> map (\ a -> a*a ) |> reduce (+) 0.0f32
    let sigma = f32.sqrt ( sigma / (r32 (ns-k2p2)) )
    let h      = t32 ( (r32 ns) * hfrac )
    in (h, ns, sigma)
  ) Yh y_errors
```

```
Y_err_sh[threadIdx.x] = threadIdx.x < red_res ? Y_err_sh[threadIdx.x] * Y_err_sh[threadIdx.x] : 0.0;
__syncthreads();
```

```
tmp = scanIncBlock<Add<ElTp_default> >(Y_err_sh, threadIdx.x);
__syncthreads();
Y_err_sh[threadIdx.x] = tmp;
__syncthreads();
```

```
if(threadIdx.x == 0 ) {
  nss[blockIdx.x] = red_res;

  ElTp_default sigma_tmp = Y_err_sh[n-1];
  sigmas[blockIdx.x] = sqrt(sigma_tmp / (float) (red_res - k2p2));

  hs[blockIdx.x] = (int) ((float)red_res * hfrac);
}
```

Kernel 6 Performance

Peru dataset	GB/s	microsecs	Speedup over cpu
futharkhpa03fl.unicph.domain (A100)	418.105	242	43.913x
Naive memcpy	1200.56	-	-

CPU in microsecs
10627

Kernel 7

- Remove loops (CPU)/maps(Futhark) and replace it with parallel access
 - Array Expansion
 - Deconstruction of operations
- Use shared memory to reduce access to global
- Compute the maximum of hs on CPU (simple Sequential search)
 - Originated from a misunderstanding
 - Values are copied back to CPU for Validation
 - In our case no performance difference
 - In general unnecessary copying of data

Kernel 7 (Compute MO_fsts)

- Outer loop is replaced by m blocks
- Inner loop for MO_fsts is replaced by an expansion of MO_fsts_sh and a scan over the results

```
let MO_fsts = zip3 y_errors nss hs |>
  map (\(y_error, ns, h) ->
    map i32.i64 (iota hmax)
    |> map (\i -> if i < h then y_error[i + ns-h+1] else 0.0)
    |> reduce (+) 0.0
  ) |> opaque
```

```
__global__ void kernel7_new(int m, int N, int n, float lam, float *y_errors, int *nss,
  int *hs, int *mapping_indecies, float *MO_fsts, float *BOUND, int h_max) {
  extern __shared__ float shmem[];
  volatile float* MO_fsts_sh = shmem;
  int id = threadIdx.x + blockIdx.x * blockDim.x;
  int h = hs[blockIdx.x];
  int ns = nss[blockIdx.x];

  MO_fsts_sh[threadIdx.x] = threadIdx.x < h ? y_errors[blockIdx.x * N + threadIdx.x + ns-h+1] : 0.0;

  __syncthreads();
  float tmp = scanIncBlock< Add<float> >>(MO_fsts_sh, threadIdx.x);
  __syncthreads();

  if(threadIdx.x == h_max - 1) {MO_fsts[blockIdx.x] = tmp;}
```

Kernel 7 (Compute MO_fsts)

Map is replaced by the first N-n blocks, accessing and computing a single element

```
let BOUND = map (\q -> let t    = n+1+q
                        let time = mappingindices[t-1]
                        let tmp  = logplus ((r32 time) / (r32 mappingindices[N-1]))
                        in  lam * (f32.sqrt tmp)
                ) (iota (N-n))

if(id < N - n) {
    int t = n + 1 + id;
    float x = (float) mapping_indecies[t-1]/(float) mapping_indecies[N-1];
    if(x > expf(1)){
        x = log(x);
    }else{
        x = 1.0;
    }
    BOUND[id] = lam * sqrt(x);
}
```

Kernel 7 (Performance)

- The speed up between the naive and optimized version is minor $\sim 1.38x$
- But we achieved a more significant speed up compared to CPU $\sim 7x$ on Peru

Naive tested on the peru dataset	microseconds
futharkhpa03fl.unicph.domain (A100)	119

CPU in microsecs
565

Optimized tested on the peru dataset	microseconds
futharkhpa03fl.unicph.domain (A100)	86
Improvements	1.38

Kernel 8

- Goal of Kernel: compute breaks (index of first break) + mean (magnitude)
- Naive version: Outer parallelism -> m blocks
- Optimized: exploit also inner-parallelism: (N-n) threads per block
 - parallelized some of the map-reduce constructs
- Further Optimizations possible

Version	Runtime μ -seconds	GB/s
CPU Sequential	4451	-
GPU Naive	1697	65.47
GPU Optimized	580	219.12
Mem-copy Kernel	-	1200.56

Kernel 8 on Peru dataset on A100

```

map ( $\lambda y \rightarrow$  let  $\bar{y}_{[:n]} = y[:n]$ 
      . . .
      let  $\bar{m}^0 = \text{map } (\lambda t \rightarrow \text{if } t == 0 \text{ then } m^{fst}
                        \text{ else } \bar{r}[\bar{n}+t] - \bar{r}[\bar{n}-h+t]
                        ) (0 \dots \bar{N} - \bar{n} - 1) \mid > \text{scan } (+) \ 0.0$ 
      let  $\bar{m} = \text{map } (\lambda m_t^0 \rightarrow m_t^0 / (\hat{\sigma} * (\text{sqrt } (r32 \ \bar{n})))
                    ) \ \bar{m}^0$ 
      let mean = reduce (+) 0.0  $\bar{m}$ 
      let (found_break, brk) =
        map2 ( $\lambda m_t \ t \rightarrow$ 
              let  $b_t = \lambda * (\text{sqrt } (\log_+(\frac{t}{\bar{n}})))$ 
              in ((abs  $m_t$ ) >  $b_t$ , t)
              )  $\bar{m}$  (0... $\bar{N} - \bar{n} - 1$ )  $\mid >$ 
        reduce (  $\lambda (b1, i1) (b2, i2) \rightarrow$ 
                if b1 then (b1, i1) else (b2, i2)
                ) (false, -1)
      let brk' = if !found_break then -1
                else remapIndices brk n  $\bar{n}$   $\bar{N}$   $\bar{I}$ 
      in (brk', mean)
      . . .
    ) Y

```

Kernel 8

```
let (breaks, means) = zip (zip4 Nss nss sigmas hs) (zip3 MO_fsts y_errors val_indss) |>
  map (\ ( (Ns,ns,sigma, h), (MO_fst,y_error,val_inds) ) ->
    let MO = map i32.i64 (indices BOUND)
    |> map (\j -> if j >= Ns-ns then 0.0
      else if j == 0 then MO_fst
      else (-y_error[ns-h+j] + y_error[ns+j]))
    )
    |> scan (+) 0.0f32

let MO' = map (\mo -> mo / (sigma * (f32.sqrt (r32 ns))) ) MO
let (is_break, fst_break) =
  map i32.i64 (indices BOUND)
  |> map3 (\mo' b j -> if j < Ns - ns && !(f32.isnan mo')
    then ( (f32.abs mo') > b, j )
    else ( false, j )
    ) MO' BOUND
  |> reduce (\ (b1,i1) (b2,i2) ->
    if b1 then (b1,i1)
    else if b2 then (b2, i2)
    else (b1,i1)
    ) (false, -1)
let mean = map i32.i64 (indices BOUND)
  |> map2 (\x j -> if j < Ns - ns then x else 0.0 ) MO'
  |> reduce (+) 0.0

let fst_break' = if !is_break then -1
  else let adj_break = adjustValInds (i32.i64 n) ns Ns val_inds fst_break
    in adj_break
let fst_break' = if ns <= 5 || Ns-ns <= 5 then -2 else fst_break'

in (fst_break', mean)
) |> unzip
```

(N-n) threads (inner-parallelism) + shared memory to perform map and scan

```
if (tid < (N-n)) {
  if (tid >= Ns-ns){
    M0[tid] = 0.0f;
  }
  else if (tid == 0){
    M0[tid] = MO_fsts[bid];
  } else{
    M0[tid] = (-y_errors[bid * N + ns-h + tid] +
      y_errors[bid * N + ns + tid]);
  }
}
__syncthreads();

M0[tid+(N-n)] = scanIncBlock< Add<f32> >(M0, tid);
__syncthreads();
M0[tid+(N-n)] = M0[tid+(N-n)] / (sigma * (sqrtf((float)ns)));
```

Performed sequentially (loops) by one thread in block (not optimal), could still parallelize map and reduce constructs

```
if (tid == 0) {
  . . .
}
```

Performance

	Ker1	Ker2	Ker3	Ker4	Ker5	Ker6	Ker7	Ker8
CPU in μ -sec	10963	130012	44837	50254	34729	15179	1163	5825
GPU in μ -sec	1276	1120	341	571	417	255	307	1040
Speed-Up	8	116	131	88	83	59	3	5

Kernel Performance Sahara dataset on A100

	Ker1	Ker2	Ker3	Ker4	Ker5	Ker6	Ker7	Ker8
CPU in μ -sec	9293	108541	73066	24144	12683	10596	565	4451
GPU in μ -sec	1021	628	408	575	331	243	86	582
Speed-Up	9	172	179	41	38	43	7	7

Kernel Performance Peru dataset on A100

Conclusion

- We achieved project goals
- Significant performance improvements
- Shortcomings
 - Big epsilon – some of the kernels don't validate, but the overall results are ok
 - Not all kernels fully optimized